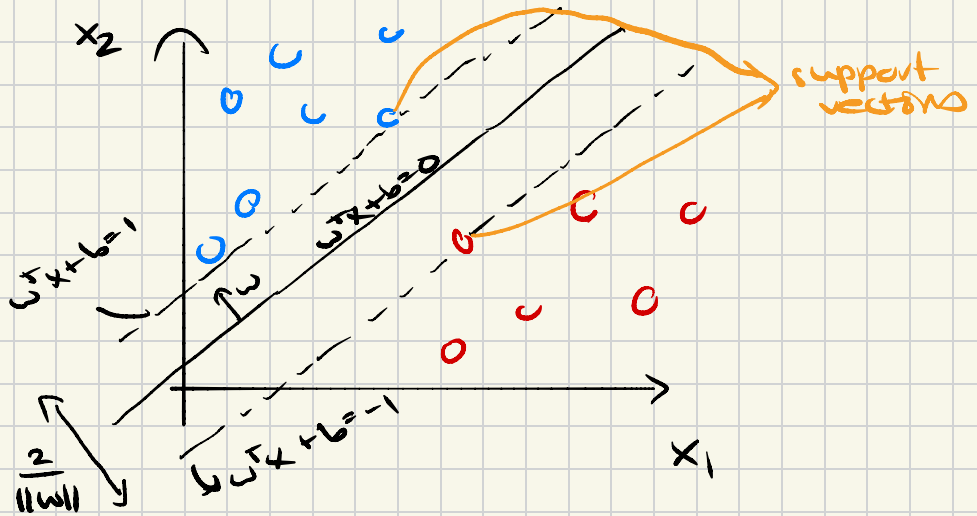




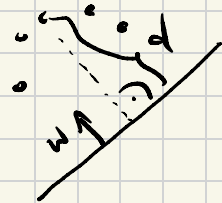
- Defined 2 add. hyperplanes:

$$w^T x + b - 1 = 0$$

$$w^T x + b + 1 = 0$$

- The perpendicular distance d of any point x from the hyperplane $w^T x + b = 0$:

$$d = \frac{|w^T x + b|}{\|w\|}$$



- For the support vectors

$$|w^T x_i + b| = 1$$

- The minimum distance from the hyperplane

is then

$$\frac{1}{\|w\|}$$

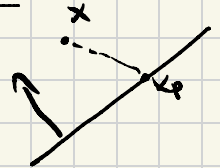
- The margin width is therefore given by

$$\frac{1}{\|w\|} + \frac{1}{\|w\|} = \frac{2}{\|w\|}$$

Understanding distance computation

Assump.

$$d = \frac{|w^T x + b|}{\|w\|}$$



- let x_p be any point on the hyperplane ($w^T x_p + b = 0$)
- The vector $x - x_p$ connects x to the hyperplane
- The perpendicular distance

$$d = \hat{w}^T (x - x_p) = \frac{w^T (x - x_p)}{\|w\|} = \frac{w^T x - w^T x_p}{\|w\|}$$

$$\hat{w} = \frac{w^T}{\|w\|}$$

- since: $w^T x_p + b = 0 \Rightarrow w^T x_p = -b$

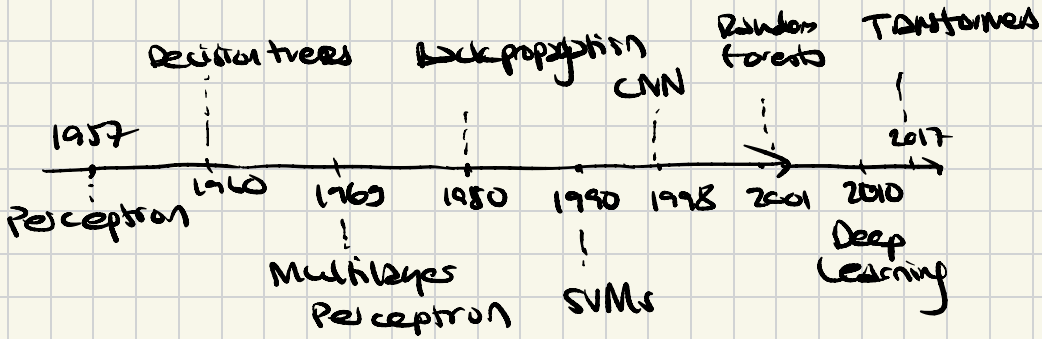
- Inserting this back, gives us the signed distance

$$d = \frac{w^T x + b}{\|w\|}$$

- Take the absolute value:

$$d = \frac{|w^T x + b|}{\|w\|}$$

Important Milestones



Primal SVM formulation

- Optimization criterion for the hard-margin SVM

$$\min_{w, b} \quad \frac{1}{2} \|w\|_2^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 \\ \text{for } i=1 \dots N$$

- Problem: inequality constraints that are annoying to enforce directly
- Idea: Convert them into penalty terms in a single unconstrained "Lagrangian" fun
- Each of the N constraints can be written as
$$g_i(w, b) = y_i(w^T x_i + b) - 1 \geq 0$$

- Then the Lagrangian is:

$$L(w, b, \alpha) = \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^N \alpha_i g_i(w, b)$$

$$= \frac{1}{2} \|w\|_2^2 - \sum_i \alpha_i [y_i(w^T x_i + b) - 1] \\ \alpha_i \geq 0$$

- what we want :

$$\min_{w, b} \max_x L(w, b, x)$$

- why does it work?

$$\max_{x_i \geq 0} \left(- \sum_i x_i \underbrace{(y_i (w^T x_i + b) - 1)}_{:= c_i} \right)$$

$$- x_i c_i = \begin{cases} \geq 0 & c_i < 0 \\ \leq 0 & c_i \geq 0 \end{cases}$$

constraint is violated maxim x_i $L \rightarrow \infty$
 constraint is satisfied maxim x_i $x_i = 0$

- The idea of the dual

- Strong duality (property of convex QPs)

means we can swap the order of min and max :

$$\min_{w, b} \max_{x \geq 0} L = \max_{x \geq 0} \min_{w, b} L$$

- First, produce

$$g(x) = \min_{w, b} L(w, b, x)$$

"dual objective"

$\max_{x \geq 0} g(x) \rightarrow$ dual problem

- minimizing L over w and b :

$$\frac{\partial L}{\partial w} = \frac{\partial}{\partial w} \left(\frac{1}{2} \|w\|_2^2 - \sum_i^N x_i (y_i (w^T x_i + b)) - 1 \right) \stackrel{!}{=} 0$$

$$= \frac{\partial}{\partial w} \left(\frac{1}{2} w^T w \right) - \sum_i^N x_i y_i \frac{\partial}{\partial w} (w^T x_i)$$

$$= w - \sum_i^N x_i y_i x_i \Rightarrow \boxed{w = \sum_i^N x_i y_i x_i}$$

$$\frac{\partial L}{\partial b} = - \sum_i^N x_i y_i \stackrel{!}{=} 0 \quad \boxed{\sum_i^N x_i y_i = 0}^*$$

- Forming our dual objective:

$$L = \underbrace{\frac{1}{2} w^T w}_A - \underbrace{\sum_i^N x_i y_i (w^T x_i)}_B - \underbrace{b \sum_i^N x_i y_i}_{C=0} + \underbrace{\sum_i^N x_i}_D$$

- compute A :

$$\begin{aligned} w^T w &= \left(\sum_i^N x_i y_i x_i \right)^T \left(\sum_j^N x_j y_j x_j \right) = \sum_i^N x_i y_i \sum_j^N x_j y_j (x_j^T x_i) \\ &= \sum_{i,j} x_i x_j y_i y_j (x_j^T x_i) \end{aligned}$$

$$A = \frac{1}{2} \quad -11-$$

- Compute B:

$$w^T x_i = \left(\sum_j \alpha_j y_j x_j \right)^T x_i = \sum_j \alpha_j y_j (x_j^T x_i)$$

$$B = \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_j^T x_i)$$

- Assembling into the dual:

$$g(\alpha) = -\frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_i^T x_j) + \sum_i \alpha_i$$

$$\alpha_i \geq 0, \quad i=1 \dots N$$

$$\sum_i \alpha_i y_i = 0$$

- Optimization:

$$\alpha^* = \underset{\alpha}{\operatorname{argmax}} g(\alpha)$$

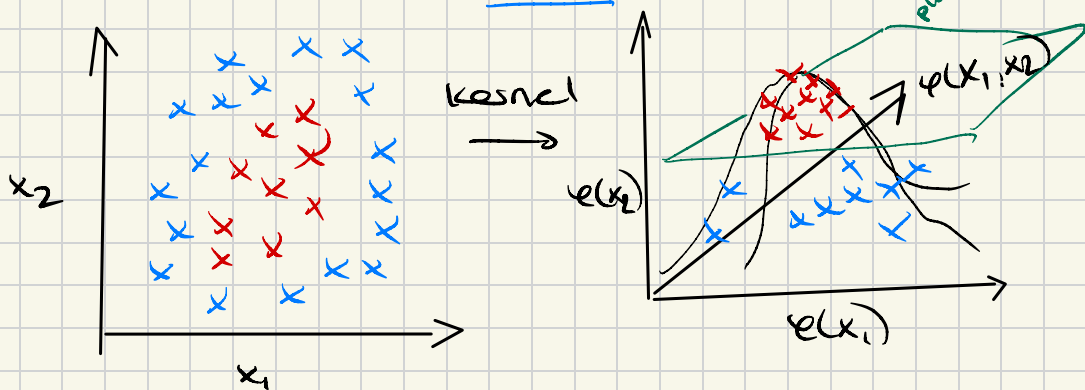
- Once we have α^* , we can compute

$$\hat{w} = \sum_i \alpha_i^* y_i x_i$$

2.5.2 KERNEL TRICK

- observation: real-world data often not linearly separable
- idea: map the input features into a higher dim. space using a nonlinear transformation $\phi(x_i)$, and then apply SVM there

EXAMPLE



- challenge: computing $\phi(x)$ explicitly can be very expensive (especially for high D spaces)
- kernel trick: instead of computing $\phi(x_i)^T \phi(x_j)$ directly, we can use a kernel function:
$$\text{compute this} \rightarrow K(x_i, x_j) = \phi(x_i)^T \phi(x_j) \leftarrow \text{not this}$$

• Example:

- Consider a polynomial kernel of degree 2:

$$K(x, x') = (\langle x, x' \rangle + 1)^2$$

$$x = [x_1, x_2] \quad x' = [x_1', x_2'] \text{. Then}$$

$$K(x, x') = (x_1 x_1' + x_2 x_2' + 1)^2$$

- This is equivalent to computing

$$\phi(x) = [x_1^2, \sqrt{2} x_1 x_2, x_2^2, \sqrt{2} x_1, \sqrt{2} x_2, 1]$$

$$\begin{aligned} \phi(x)^T \phi(x') &= x_1^2 x_1'^2 + 2x_1 x_2 x_1' x_2' + x_2^2 x_2'^2 \\ &\quad + 2x_1 x_1' + 2x_2 x_2' + 1 \end{aligned}$$

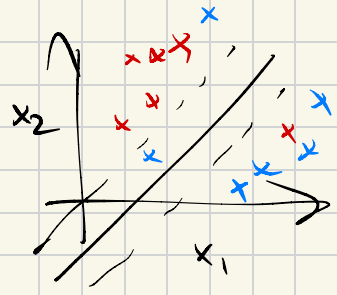
• Common kernels

- Polynomial kernels: $K(x_i, x_j) = (x_i^T x_j + c)^d$

- Radial basis functions:

$$K(x_i, x_j) = \exp\left(-\underset{\substack{\text{hyperparam} \\ \downarrow}}{\gamma} \|x_i - x_j\|^2\right)$$

2.5.3 Soft Margin SVMs



- For non-separable data, we introduce "slack variables"

$\xi_i \geq 0$ to allow for some misspecification

- For each training example (x_i, y_i) , the soft margin constraints look like:

$$y_i (w^T x_i + b) \geq 1 - \xi_i \quad \xi_i \geq 0$$

- $y_i (w^T x_i + b) \geq 1 \Rightarrow \xi_i = 0$

- $y_i (w^T x_i + b) < 1 \Rightarrow \xi_i = 1 - y_i (w^T x_i + b)$

- The soft-margin optimization problem:

$$\min_{w, b, \xi_i} \left[\frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^N \xi_i \right]$$

$$\text{s.t. } y_i (w^T x + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- large C :

→ strong penalization slack variables (error)

→ narrower margin

- small C :

→ allows for more errors

→ produces wider margin

• connecting loss to the "Hinge loss" (ReLU)

$$\begin{aligned} L_{\text{Hinge}} &= \sum_{i=1}^N \max(0, 1 - y_i(w^T x_i + b)) \\ &= \sum_{i=1}^N \text{ReLU}(1 - y_i(w^T x_i + b)) \end{aligned}$$

• Finally, our optimization criterion:

$$\min_{w, b} \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^N \text{ReLU}(1 - y_i(w^T x_i + b))$$

Summarize

• Perceptron

• iterative algo. for binary classif.

• converges if data is lin. separable

• simple update rule (Gradient Descent)

• SVMs

• robust classification by optimizing margin

• can handle non-separable data

• dual formulation leads to the kernel trick

2.6 Cross-validation AND Error Analysis

- Observation: Zero training error does not imply "a good classifier" $\hat{=}$ low generalization error
- Definition: Generalization error

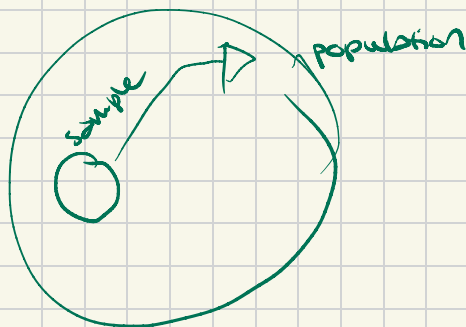
$$\text{Err}_1 = \mathbb{E}_{(x,y) \sim p^*(x,y)} \left[\underbrace{\Pr(\text{error}(x,y) | \hat{f}(x))}_{\hat{f}(x) \neq y} \right]$$

$$= \mathbb{E}_{(x,y) \sim p^*(x,y)} \left[\underbrace{1\{\hat{f}(x) \neq y^*\}}_{= 1 \text{ if I make an error in predict.}} \right]$$

in case of deterministic classifiers /
prob. classifiers with hard decision

in case of prob. classifiers, with soft decisions:

$$\text{Err} = \mathbb{E}_{(x,y) \sim p^*(x,y)} \left[1 - \Pr(\hat{y} = y^* | x) \right]$$



26.1 Training Error vs. Test Error

- setting: observed a dataset $D = \{(x_i, y_i)\}_{i=1}^N$
training set

$$\begin{aligned} \text{err} &:= \mathbb{E}_{(x,y) \sim D} [\text{p(error} | \text{current algo.})] \\ &= \frac{1}{|D|} \sum_{(x_i, y_i) \in D} \mathbb{1}\{\hat{f}(x) \neq y_i\} \\ &= \text{training error} \end{aligned}$$

changed as long as err reduces

- $\text{err} < \text{Err}$
- The difference:

$$\text{optimism} := \text{Err} - \text{err}$$

- if optimism is large $\rightarrow$ indicates overfitting
 $\Rightarrow$ err is not! a good indicator of Err!

Approaches to estimate gen. error:

- 1) Exact analytical analysis (feasible for toy problems)
- 2) Asymptotic Analysis (results $N \rightarrow \infty$)
- 3) Empirical Estimation

2.6.2 Empirical Error Estimation via (k-fold) cross-validation (CV)

Procedure:

- 1) Split the training set (TS) into M folds of roughly equal size $(\frac{N}{M})$
- 2) For each $m = 1 \dots M$:
 - use fold m as test set.
 - Train the model on remaining data
 - compute the test error for fold m :

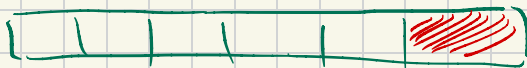
$$\hat{\text{Err}}_m = \frac{\# \text{ errors in fold } m}{N_m}$$

↪ size of fold m

- 3) compute the overall CV error:

$$\hat{\text{Err}} = \frac{1}{M} \sum_{m=1}^M \hat{\text{Err}}_m$$

1st iter.



2nd iter



⋮

M^{th} iter



$$\left. \begin{array}{l} \hat{\text{Err}}_1 \\ \hat{\text{Err}}_2 \\ \vdots \\ \hat{\text{Err}}_M \end{array} \right\} \begin{array}{l} \text{average for} \\ \hat{\text{Err}} \end{array}$$

How to obtain a classifier from this procedure?

- 1) Retrain model on entire data set
- 2) Use all classifiers (ensemble classific.)
(majority vote class.)

How to obtain $\hat{f}(x)$?

2.7 Three core paradigms to param. inference

1) Empirical Risk Minimization (ERM)

- define a loss function $\ell(\tilde{y}, \hat{y})$
- minimize average loss across obs. data pts.

$$\hat{\theta}_{\text{ERM}} = \underset{\theta}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^N \ell(y_i, \underbrace{\hat{f}_{\theta}(x_i)}_{\hat{y}_i})$$

- Most commonly: Method of Least Squares

$$\hat{\theta}_{\text{LS}} = \underset{\theta}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^N (y_i - \hat{f}_{\theta}(x_i))^2$$

usually

- deterministic, no assumptions on prob. distr.

2) Maximum likelihood estimation (MLE)

- assume a model for the data-generating process, typically specified by cond. likelihood

$$\hat{\theta}_{MLE} = \underset{\theta}{\operatorname{argmax}} p(y_1, \dots, y_N | x_1, \dots, x_N; \theta)$$

3) Bayesian inference

- treats parameters as random variables

$$p(\theta | D) = \frac{p(D | \theta) p(\theta)}{p(D)}$$

2.3.1 Probability Theory and Bayes Rule

- chain rule of probability

$$p(x, y) = p(x) p(y | x) = p(y) p(x | y)$$

- extension to 3 variables:

$$p(x, y, z) = p(x) p(y | x) p(z | x, y) = \dots$$

• Bayes Rule:

$$\underbrace{p(Y|X)}_{\text{posterior}} = \frac{\underbrace{p(X|Y)}_{\text{likelihood}} \underbrace{p(Y)}_{\text{prior}}}{\underbrace{p(X)}_{\text{marginal}}}$$

$$\sum_k p(Y=k|X) = 1$$

$$\Rightarrow p(X) = \sum_k p(X|Y=k) p(Y=k)$$

• Example: prob. of obs. ghost give a knock

$G = \text{ghost}$

$\neg G = \text{no ghost}$

$K = \text{knock}$

$$p(G|K) = \frac{p(K|G) p(G)}{p(K)}$$

2.2.2 Worst-Case Performance of Probabilistic Classifier

• If features X are uninformative for Y , then

$$p(X|Y=k) = p(X|Y=k') \quad \forall k, k'$$

• inserting into Bayes law:

$$\begin{aligned} p(Y=k|X) &= \frac{p(X|Y=k)p(Y=k)}{\sum_k p(X|Y=k)p(Y=k)} \\ &= \frac{\cancel{p(X|Y=k)}p(Y=k)}{\cancel{p(X|Y=k)} \underbrace{\sum_k p(Y=k)}_{=1}} \\ &= p(Y=k) \quad \text{posterior} = \text{prior} \end{aligned}$$

• what to do? Decision implication:

- if $p(Y=0) < p(Y=1)$ always predict $\hat{Y}=1$
- if $p(Y=0) = p(Y=1) = 0.5 \Rightarrow$ guess randomly
- classifier should do no worse than random guessing ($\frac{1}{C}$ accuracy for C classes)

2.3.3 Discriminative vs. generative classifiers

- 1) Discriminative classif.: learns posterior $p(Y|X)$ directly
- 2) Generative classif.: learns prior $p(Y)$ and likelihood $p(X|Y)$, then uses Bayes rule to obtain posterior

2.7.4 Maximum Likelihood Principle

- setting: observed TS = $\{x_i\}_{i=1}^N$
- assumption: TS is a typical representation of CW data
- The principle:
 - since we cannot adjust data, we adjust param. :

$$\max_{\theta} p(\text{TS} | \theta) = \max_{\theta} \psi(x_1, \dots, x_N | \theta)$$

- equivalently, minim. log-likelihood:

$$\hat{\theta}_{MLE} = \arg\min_{\theta} [-\log p(\text{TS} | \theta)]$$

The i.i.d. Assumption

- Data/TS is said to be independent and identically distributed (i.i.d.)

1) Each training instance is drawn indep.

$$p(\text{TS}) = p(\{x_i, y_i\}_{i=1}^N) = \prod_{i=1}^N p(x_i, y_i)$$

2) All instances come from same distr.

$$p(x_i, y_i) = p^*(x, y) \quad \forall i$$

Probabilistic Classification

2.8 Generative Classification with Quadratic Discriminant Analysis

- The Nearest Mean Classify
- Idea: Represent each class by a single representative, the class mean
- For each class $k = 1 \dots C$:

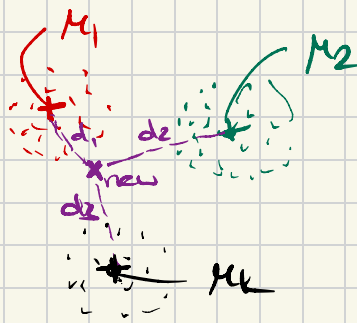
N_k : number of instances in class k

$$\mu_k = \frac{1}{N_k} \sum_{i: y_i = k} x_i \quad x_i \in \mathbb{R}^D$$

- Classification Rule:

Given a query x (new data point), assign

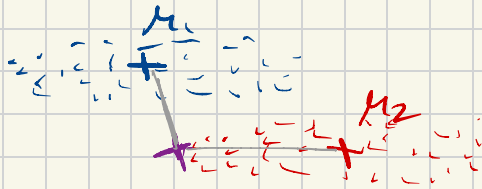
$$\hat{y} = \underset{k}{\operatorname{argmin}} \underbrace{d(x, \mu_k)}_{\text{distance metric}}$$



- Problem: Does not take into account cluster shape

Recap

- we want $\hat{\theta}$ for $\hat{y} = f_{\theta}(x)$
- 3 methods for param. estimation:
 - 1) LS (deterministic) → perceptron
→ SVM
 - 2) MLE } probabilistic
 - 3) BI }
- Problem: high dim. prob. distributions
 → make use of iid data
- Optimization is on training set, but
 what we want: generalization to pop.



2.8.1 Accounting for Cluster Shape

Motivation: when clusters are not circular, the mean is not! a sufficient representative

Form a circle to ellipse:

1) scaling:

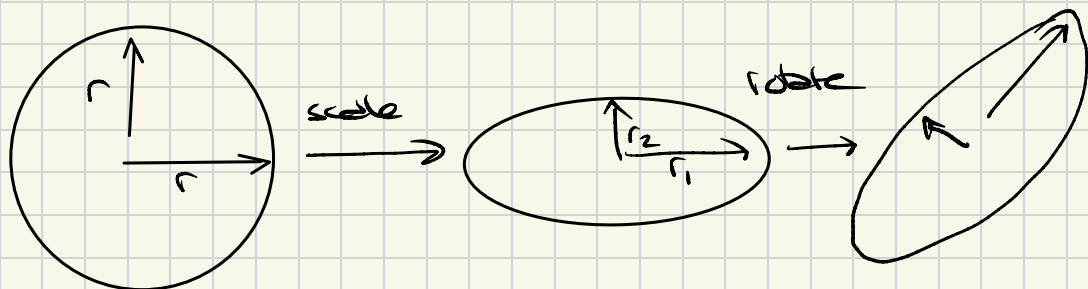
- in a circle, the radius is the same along all axes
- in an ellipse, scaling is represented by

$$\lambda = \begin{bmatrix} r_1 & 0 \\ 0 & r_2 \end{bmatrix}$$

2) rotation: using $R = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}$

combine and obtain

$$\tilde{x} = \underbrace{R \lambda}_{\Sigma \in \mathbb{R}^{D \times D}} x$$



• Differences in degrees of freedom:

- 1) circle: center (1 parameter; $x \in \mathbb{R}^D$),
and Radius (1 param): $D+1$ TOTAL $D+1$
- 2) ellipsoid: center (1), + translation scaling + rotation
by a symmetric $D \times D$ matrix: $D \frac{(D+1)}{2}$ param.
needed. (+ D for translation) TOTAL $D + \frac{D(D+1)}{2}$

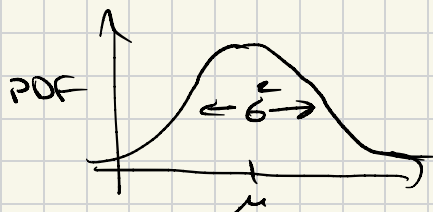
2.8.2 Generative Discriminative Analysis

(GDA) is a Generative Classifier

• Gaussian Distributions (Reminder)

Univariate

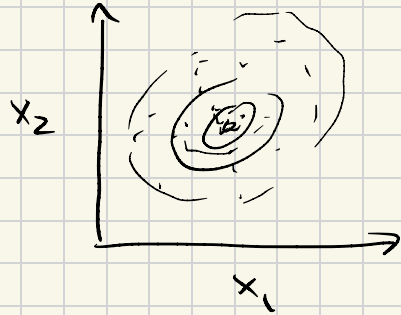
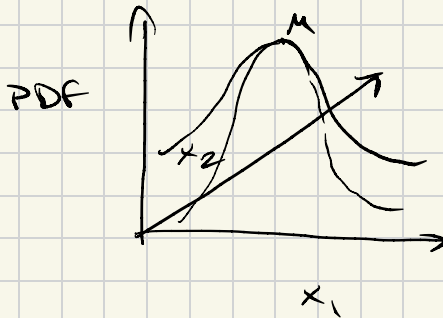
$$\mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2} \frac{(x-\mu)^2}{\sigma^2}}$$



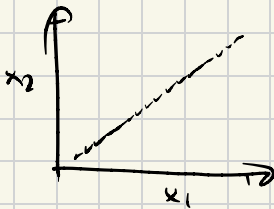
Multivariate Gaussian:

$$N(\mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^D} |\Sigma|} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)}$$

$\in \mathbb{R}^D$ $\in \mathbb{R}^{D \times D}$ $\downarrow$ determinant

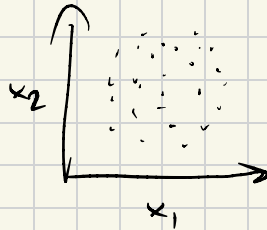


Examples of Σ

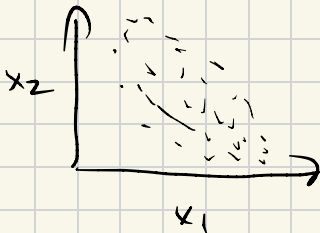


$$\Sigma = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$\rightarrow$ capture correlation in data



$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$



$$\Sigma = \begin{bmatrix} 1 & -.8 \\ -.8 & 1 \end{bmatrix}$$

GDA Idea:

- Model each class's features as following a multivariate Gaussian:

$$\text{For class } k: p(x | y=k) = \mathcal{N}(\mu_k, \Sigma_k)$$

- To obtain the generative classifier, we apply

Bayes rule:

$$p(y=k | x) = \frac{p(x | y=k) p(y=k)}{p(x)}$$

Derivation

- For class k , assume the model

$$p(x_i | \mu_k, \Sigma_k) = \frac{1}{\sqrt{(2\pi)^d |\Sigma_k|}} e^{-\frac{1}{2} (x_i - \mu_k)^T \Sigma_k^{-1} (x_i - \mu_k)}$$

- The likelihood of the training data set

(for one class) using i.i.d assumptions is:

$$\mathcal{L}_x(\mu_k, \Sigma_k) = p(x_1, \dots, x_{N_k} | \mu_k, \Sigma_k) \stackrel{\text{i.i.d } N_k}{=} \prod_{i=1}^{N_k} p(x_i | \mu_k, \Sigma_k)$$

prod. across all instances in class k

- Taking the negative logarithm (to obtain loss):
(dropped k for convenience:)